

SAP-1 — Material de Estudo Completo

Processador SAP-1 em Verilog · Quartus Prime + DE10-Lite

Compilado de todos os guias de estudo do projeto, na ordem recomendada de leitura: do panorama geral até os exercícios de fixação.

Índice

Roteiro 1. Guia de estudo (por onde começar)

Teoria — como o processador funciona 2. Componentes (cada bloco explicado) 3. A palavra de controle (CON) na prática 4. Como tudo funciona junto (barramento + 1 instrução) 5. A máquina de estados (FSM) por dentro 6. Diagrama — anel de estados 7. Diagrama — ciclo de execução por instrução 8. Diagrama — trace do exemplo

Prática — ver e fazer 9. Guia de leitura das ondas (ModelSim) 10. Um programa completo, explicado (a expressão = 18) 11. Lista de exercícios (com gabarito) 12. Roteiro de apresentação

Apêndices — consulta rápida 13. Cola (resumo de 1 página) 14. Glossário

Gerado a partir dos arquivos `.md` do repositório. Cada seção também existe como arquivo individual nas pastas `docs/` e `fsm/`.



Guia de estudo — SAP-1

Este é o **mapa** para estudar o projeto na ordem certa. Cada item tem um documento próprio; aqui você vê **o que ler, em que ordem e por quê**.

Objetivo final: entender como um processador simples **busca e executa instruções** — e conseguir ler a simulação (as ondas) com confiança.

A trilha (leia nesta ordem)

Etapa 1 — As peças

`componentes.md` → o que é cada bloco (PC, MAR, IR, A, B, ALU, RAM...).

Comece aqui. Você não precisa decorar — só entender **o que cada bloco guarda e faz**. A sacada: 5 registradores (MAR, IR, A, B, saída) são o *mesmo circuito* (carrega/segura/zera).

Você terminou quando souber dizer, de cada bloco: "isso guarda X e serve pra Y".

Etapa 2 — Como as peças são comandadas

`palavra_controle.md` → os 12 sinais de controle (a palavra CON).

Aqui você entende **como o processador liga/desliga cada bloco** a cada passo. O truque: comparar cada palavra com o "repouso" (IDLE).

Você terminou quando conseguir olhar `0101_1110_0011` e dizer "isso é MAR ← PC".

Etapa 3 — Tudo junto (o coração)

`fluxo_completo.md` → o barramento, a história de uma instrução ponta a ponta, como o controlador decide, e um **exercício**.

Este é o **documento mais importante**. Ele conecta as etapas 1 e 2 na "história" de uma instrução real (`ADD 11`), estado por estado. Faça o **exercício do 5 + 2** — é o que fixa de vez.

Você terminou quando conseguir contar, com suas palavras, o que acontece do T1 ao T6 de um `ADD`.

Etapa 4 — Ver funcionando (simulação)

`guia_ondas.md` → como ler a janela de ondas do ModelSim.

Agora você abre a simulação e **confirma com os próprios olhos** tudo que leu. Responda as **7 perguntas de autoteste** no fim do guia.

Você terminou quando responder as 7 perguntas olhando a onda.

Etapa 5 — Um programa real completo

`programa2_explicado.md` → o programa da RAM explicado instrução por instrução (a expressão = 18).

Aqui você vê **um programa inteiro** rodando, não só uma instrução. Ótimo para juntar tudo.

Etapa 6 (aprofundamento) — A máquina de estados

`FSM_explicacao.md` → o controlador por dentro (contador de anel, HLT, reset). Leia se quiser entender **como** a palavra de controle é gerada.

Etapa 7 (opcional) — Apresentar

`roteiro_video.md` → roteiro do que falar mostrando a simulação.

A parte prática (faça, não só leia)

Estudar processador **olhando** só funciona até certo ponto. O que trava o conhecimento é **fazer**:

1. **Rode a simulação** e siga o acumulador: `tcl cd tb_model do wave_sap1.do`
2. **Faça o exercício 5 + 2** (está no `fluxo_completo.md`): monte o binário, trace o A na mão, e confira na simulação.
3. **Invente um programa seu** (ex.: `4 + 4 + 4`) e preveja o resultado antes de simular.
4. **Rode um componente isolado** para ver ele sozinho:
`tcl do run_one.do accumulator do run_one.do controller_sequencer`

Checklist final — "eu entendi o SAP-1 se..."

- [] ...sei dizer o que cada bloco (PC, MAR, IR, A, B, ALU) faz.
- [] ...entendo que só **um** bloco fala no barramento por vez.
- [] ...sei que toda instrução tem **6 estados** (T1-T3 busca, T4-T6 execução).
- [] ...consigo ler uma palavra de controle e dizer "quem fala, quem ouve".
- [] ...entendo por que `LDA 11` carrega 3 (endereço ≠ valor).
- [] ...sei por que o resultado só muda no `OUT` e trava no `HLT` (T4).

- [] ...consigo montar um programa novo e prever o resultado.
- [] ...consigo abrir a onda e confirmar tudo isso.

Se marcar todos, **você entendeu o processador**.

Referência rápida

Instruções: LDA=0000 ADD=0001 SUB=0010 OUT=1110 HLT=1111 (formato: [opcode(4) | operando(4)] ; o operando é um **endereço**).

Estados: T1 T2 T3 = busca (igual p/ todas) · T4 T5 T6 = execução.

Comandos ModelSim (na pasta tb_model): | Comando | Faz | |---|---| | do wave_sap1.do | simulação completa com ondas | | do wave_controller.do | ondas do controlador | | do run_one.do <nome> | um componente com ondas | | do run_all_tb.do | todos os testes (PASSOU/FALHOU) |

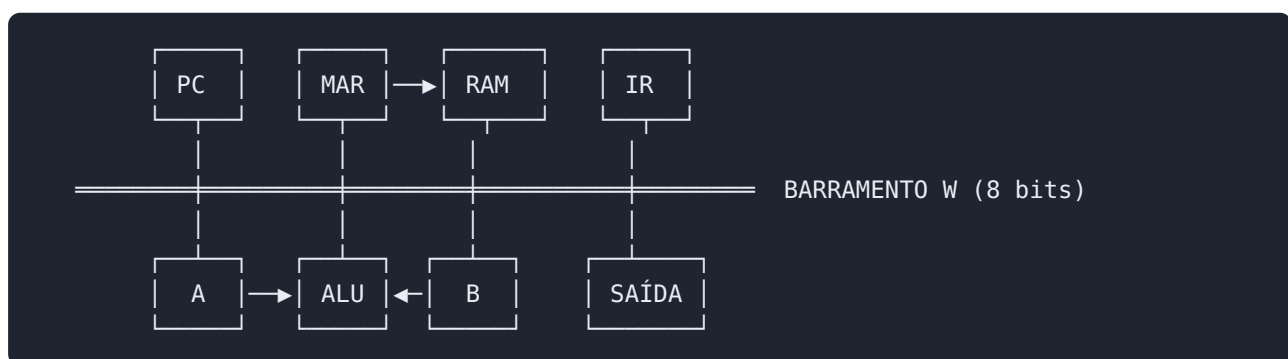
Trocar o programa: cole um programaN.txt no ram_16x8.v e ajuste o RESULTADO_ESPERADO no tb_model/tb_sap1.v.

Componentes do SAP-1 — explicação detalhada

Guia de estudo de **cada bloco** do processador SAP-1, com o código e o papel de cada um. Serve de referência para entender o projeto módulo a módulo.

Visão geral (o datapath)

Todos os blocos conversam por um único fio de 8 bits: o **barramento W**. Em cada estado (T1-T6), o controlador liga **um** bloco para "falar" no barramento e **um ou mais** para "ouvir".



O "padrão registrador" (leia isto primeiro)

Cinco blocos — **MAR, IR, Acumulador A, Registrador B e Registrador de Saída** — são o **mesmo circuito**: um registrador que **carrega do barramento quando habilitado** e **zera no reset**. Muda só a largura e o nome do sinal de carga.

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)           // reset assíncrono (ativo em baixo)
        q <= 0;             // zera
    else if (!load_bar)      // sinal de carga ativo (em baixo)
        q <= bus_in;        // copia o barramento
    // senão: mantém o valor (não faz nada)
end
```

Três comportamentos, em ordem de prioridade: 1. **Reset** (`clr_bar=0`): zera na hora, sem esperar o clock. 2. **Carrega** (`load_bar=0`): na subida do clock, copia o barramento. 3. **Segura**: se nenhum dos dois, mantém o valor guardado.

"Ativo em baixo" (o til `~` no nome, ex.: `~La`) significa que o sinal **age quando vale 0**. É a convenção do SAP-1 para os sinais de carga.

Agora, cada bloco em detalhe.

1. Program Counter (PC) — `program_counter.v`

O que é: um **contador de 4 bits** que guarda o endereço da **próxima instrução**. Vai de 0 a 15 e volta a 0.

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar) pc_out <= 4'b0000;    // reset -> 0
    else if (cp) pc_out <= pc_out + 1;    // Cp ativo -> incrementa
end
```

- **cp** (Cp): quando 1, o PC **incrementa** na próxima subida do clock. Diferente dos registradores, o PC não "carrega do barramento" — ele só **conta**.
- No **T2** de toda instrução o **cp** fica ativo → o PC já aponta para a instrução seguinte antes mesmo de executar a atual.
- É de **4 bits** porque a memória só tem 16 posições (0-15).

Papel: dizer "qual é a próxima instrução". No T1, o valor do PC é copiado para o MAR para buscar essa instrução.

2. MAR (Memory Address Register) — `mar.v`

O que é: um registrador de **4 bits** que guarda o **endereço que a RAM vai ler**. É o "dedo apontando" para uma posição da memória.

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar) addr_out <= 4'b0000;
    else if (!lm_bar) addr_out <= bus_in;    // ~Lm ativo -> carrega
end
```

- **lm_bar** (~Lm): quando 0, o MAR **carrega** os 4 bits que estão no barramento.
- A saída **addr_out** vai direto para a entrada **addr** da RAM.

Por que existe? A RAM precisa de um endereço **estável** para ler. O MAR "segura" esse endereço enquanto a RAM entrega o dado. Ele é carregado duas vezes por instrução: - **T1**: recebe o PC (para buscar a **instrução**). - **T4**: recebe o operando do IR (para buscar o **dado**).

3. Instruction Register (IR) — `instruction_register.v`

O que é: guarda a **instrução** de 8 bits lida da RAM e a **divide em dois pedaços de 4 bits**.

```
reg [7:0] ir;
always @(posedge clk or negedge clr_bar) begin
```

```

    if (!clr_bar)      ir <= 8'b0;
    else if (!li_bar)  ir <= bus_in;    // ~Li ativo -> carrega a instrução
end
assign opcode = ir[7:4];    // nibble alto = qual instrução
assign operand = ir[3:0];    // nibble baixo = endereço do operando

```

- **li_bar** (~Li): quando 0, carrega a instrução (feito no **T3**).
- **opcode** (bits 7-4): diz **o que fazer** (LDA/ADD/SUB/OUT/HLT). Vai para o **controlador**.
- **operand** (bits 3-0): diz **onde está o dado** (um endereço). Vai para o barramento (via **~Ei**) e daí para o MAR no T4.

Papel: é a "instrução em execução". O controlador olha o **opcode** para decidir os próximos passos (T4-T6).

4. Acumulador A — `accumulator.v`

O que é: o registrador de **8 bits** mais importante — é **onde a conta acontece**. Quase toda instrução mexe nele.

```

always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)      acc_out <= 8'b0;
    else if (!la_bar)  acc_out <= bus_in;    // ~La ativo -> carrega A
end

```

- **la_bar** (~La): quando 0, A carrega o valor do barramento.
- A saída **acc_out** faz **duas coisas ao mesmo tempo**: 1. alimenta **continuamente** a ALU (entrada **acc**); 2. vai para o barramento quando **Ea** está ativo (ex.: no **OUT**).

Quem escreve em A? - **LDA**: $A \leftarrow \text{dado da RAM (no T5)}$. - **ADD / SUB**: $A \leftarrow \text{resultado da ALU (no T6)}$.

É por isso que, no 3×4, você vê **acc_out** indo $3 \rightarrow 6 \rightarrow 9 \rightarrow 12$.

5. Registrador B — `register_b.v`

O que é: registrador de **8 bits** que guarda o **segundo operando** da soma/subtração. A ALU sempre calcula $A \pm B$, então B precisa segurar o outro número.

```

always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)      b_out <= 8'b0;
    else if (!lb_bar)  b_out <= bus_in;    // ~Lb ativo -> carrega B
end

```

- **lb_bar** (~Lb): quando 0, carrega B (feito no **T5** de ADD/SUB).
- A saída **b_out** alimenta a ALU continuamente (entrada **breg**).

Papel: no **ADD 11**, o T5 carrega B com o dado (3); no T6 a ALU faz $A + B$ e o resultado volta para A.

6. ALU (somador/subtrator) — `adder_subtractor.v`

O que é: a **calculadora**. É **combinacional** (sem clock): calcula na hora, o tempo todo.

```
assign result = su ? (acc - breg) : (acc + breg);
```

- `su` (Su): seletor da operação.
- `su = 0` → `result = A + B` (soma)
- `su = 1` → `result = A - B` (subtração, por complemento de dois)
- As entradas `acc` e `breg` vêm direto dos registradores A e B.
- A saída `result` vai para o barramento quando `Eu` está ativo (no T6).

Importante: a ALU **não guarda nada**. Ela sempre mostra `A ± B` do momento. Por isso, na onda, `result` fica "mexendo" conforme A e B mudam — é normal; o que importa é o valor no instante em que `Eu` joga no barramento.

Como é 8 bits, há **overflow** (dá a volta): `200 + 100 = 44` (`300 - 256`).

7. Registrador de Saída — `output_register.v`

O que é: guarda o **resultado** para mostrar nos LEDs/displays. É o único jeito do processador "falar" com o mundo.

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)    out_port <= 8'b0;
    else if (!lo_bar) out_port <= bus_in;    // ~Lo ativo -> carrega saída
end
```

- `lo_bar` (~Lo): quando 0, carrega o valor (só na instrução `OUT`).
- Entre dois `OUT`, ele **congela** — por isso, na onda, `out_port` fica parado até o próximo `OUT`.

Papel: no `OUT`, o acumulador é copiado para cá (A → barramento → saída), e o valor aparece no visor.

8. RAM 16×8 — `ram_16x8.v`

O que é: a **memória** — 16 posições de 8 bits, guardando programa e dados. Só leitura (o SAP-1 não tem `STA`).

```
reg [7:0] mem [0:15];           // 16 gavetas de 8 bits
initial begin ... end           // grava o programa (vira ROM na FPGA)
assign data_out = mem[addr];     // leitura combinacional
```

- `addr` (4 bits, vem do MAR): qual posição ler.

- **data_out** (8 bits): o conteúdo, que vai para o barramento (via **~CE**).
- Cada palavra é **[opcode(4) | operando(4)]** se for instrução, ou um número puro se for dado.

Mapa: posições **0-10** = programa; **11-15** = dados. Ex.: **mem[11] = 3**, por isso **LDA 11** carrega 3.

9. Decodificadores de display — **seg7.v** e **seg7_instr.v**

seg7 — converte um valor de 4 bits (0-F) nos 7 segmentos do display (ativo em baixo: 0 acende o segmento).

```
case (val)
  4'h0: seg = 7'b1000000; // acende todos menos o 'g' -> "0"
  ...
endcase
```

seg7_instr — igual, mas converte o **opcode** numa **letra** para mostrar a instrução: **LDA→L**, **ADD→A**, **SUB→5**, **OUT→o**, **HLT→H**.

Esses dois só existem para a **placa** (não afetam a lógica do processador).

10. Blocos de placa — **clock_divider.v** e **debouncer.v**

Só entram no topo de FPGA (**sap1_fpga.v**), não no núcleo:

- **clock_divider**: divide os 50 MHz da placa para **~1 Hz**, para você ver a execução acontecer devagar (**freq = 50MHz / (2·DIV)**).
- **debouncer**: filtra o "ruído" mecânico do botão, para cada apertado do **KEY[1]** gerar **um** pulso limpo de clock (modo passo a passo).

Tabela-resumo dos sinais de controle

Bloco	Sinal de carga/ação	Ativo em	Quando age
PC	cp (Cp)	alto	T2 (incrementa)
MAR	lm_bar (~Lm)	baixo	T1 e T4
RAM	ce_bar (~CE)	baixo	T3 e T5 (lê)
IR	li_bar (~Li)	baixo	T3
A	la_bar (~La)	baixo	T5 (LDA), T6 (ADD/SUB)
B	lb_bar (~Lb)	baixo	T5 (ADD/SUB)
ALU	su (Su) / eu (Eu)	alto	T6
Saída	lo_bar (~Lo)	baixo	T4 (OUT)

O que **decide** ligar cada sinal em cada estado é o **controlador/sequenciador** — explicado em `FSM_explicacao.md`.

A palavra de controle (CON) — na prática

Guia de estudo do sinal mais importante do controlador do SAP-1: a **palavra de controle** de 12 bits (`con`). Ela é a "receita" de cada estado.

1. O que é: 12 interruptores, um por sinal

A cada estado (T1–T6), o controlador gera **12 bits de uma vez** — cada bit **liga ou desliga** um sinal de controle. É um painel de 12 interruptores que muda a cada estado.

```
CON = { Cp, Ep, ~Lm, ~CE, ~Li, ~Ei, ~La, Ea, Su, Eu, ~Lb, ~Lo }  
bit:  11 10  9  8  7  6  5  4  3  2  1  0
```

Cada bit comanda um bloco do processador (ligar a saída no barramento, ou mandar carregar):

Bit	Sinal	O que faz quando ativo
11	Cp	PC incrementa (+1)
10	Ep	PC joga seu valor no barramento
9	~Lm	MAR carrega do barramento
8	~CE	RAM joga o dado no barramento
7	~Li	IR carrega do barramento
6	~Ei	IR joga o operando (endereço) no barramento
5	~La	Acumulador A carrega do barramento
4	Ea	A joga seu valor no barramento
3	Su	ALU subtrai (em vez de somar)
2	Eu	ALU joga o resultado no barramento
1	~Lb	Registrador B carrega do barramento
0	~Lo	Registrador de saída carrega do barramento

2. A pegadinha: nem todo "ativo" é 1

Há dois tipos de sinal:

Tipo	Ativo quando	Quais
Ativo-alto (sem til)	vale 1	Cp, Ep, Ea, Su, Eu
Ativo-baixo (com til ~)	vale 0	~Lm, ~CE, ~Li, ~Ei, ~La, ~Lb, ~Lo

Por isso o estado **parado** (IDLE), onde ninguém faz nada, **não é** `0000...`. É:

```
IDLE = 0011_1110_0011
```

Os ativos-alto ficam **0** (desligados) e os ativos-baixo ficam **1** (desligados). Guarde esse valor: é o "repouso".

3. Decodificando um de verdade: T1

No código: `con = 12'b0101_1110_0011`. Abrindo bit a bit:

```
Cp Ep ~Lm ~CE ~Li ~Ei ~La Ea Su Eu ~Lb ~Lo
valor: 0 1 0 1 1 1 1 0 0 0 1 1
      | | |
      | | └─ ~Lm = 0 -> ATIVO! (MAR carrega)
      | └─── Ep = 1 -> ATIVO! (PC joga no barramento)
      └───── Cp = 0 -> desligado
```

Resultado: Ep liga a saída do PC no barramento e ~Lm manda o MAR carregar → **MAR ← PC**. Só esses dois; o resto em repouso.

4. O truque de leitura: compare com o IDLE

```
IDLE = 0011_1110_0011
T1   = 0101_1110_0011
      ↑↑
      diferem em Ep (0→1) e ~Lm (1→0)
```

Para ler qualquer CON, compare com o IDLE. Os bits que **diferem** são exatamente os sinais ativos naquele estado. É o jeito mais rápido de entender uma palavra de controle.

5. Mais exemplos reais

T3 — `0010_0110_0011` (busca da instrução):

```
~CE = 0 (ATIVO: RAM joga no barramento)
~Li = 0 (ATIVO: IR carrega)
-> IR ← RAM
```

T6 do ADD — `0011_1100_0111` (a soma):

```
~La = 0 (ATIVO: A carrega)
Eu = 1 (ATIVO: ALU joga no barramento)
-> A ← (A + B)
```

T6 do SUB — `0011_1100_1111`: igual ao ADD, mas com `Su = 1` também → a ALU subtrai → **A ← (A - B)**.

6. Como isso aparece na onda

Na janela Wave, o sinal `con` (deixado em **binário**) mostra esses 12 bits mudando a cada estado. Leia assim:

1. Olhe o `tstate` (ex.: T1).
2. Olhe o `con` naquele instante (ex.: `0101_1110_0011`).
3. Compare com o IDLE → os bits diferentes são os sinais ativos.
4. Confira: os sinais individuais (`ep`, `lm_bar` ...) logo abaixo do `con` batem com a sua leitura.

7. Tabela completa (todas as receitas)

Estado	CON (binário)	Sinais ativos	Ação
IDLE	<code>0011_1110_0011</code>	—	nada
T1	<code>0101_1110_0011</code>	<code>Ep</code> , <code>~Lm</code>	<code>MAR ← PC</code>
T2	<code>1011_1110_0011</code>	<code>Cp</code>	<code>PC ← PC+1</code>
T3	<code>0010_0110_0011</code>	<code>~CE</code> , <code>~Li</code>	<code>IR ← RAM</code>
T4 (LDA/ADD/SUB)	<code>0001_1010_0011</code>	<code>~Lm</code> , <code>~Ei</code>	<code>MAR ← operando</code>
T4 (OUT)	<code>0011_1111_0010</code>	<code>Ea</code> , <code>~Lo</code>	<code>Saída ← A</code>
T5 (LDA)	<code>0010_1100_0011</code>	<code>~CE</code> , <code>~La</code>	<code>A ← RAM</code>
T5 (ADD/SUB)	<code>0010_1110_0001</code>	<code>~CE</code> , <code>~Lb</code>	<code>B ← RAM</code>
T6 (ADD)	<code>0011_1100_0111</code>	<code>~La</code> , <code>Eu</code>	<code>A ← A+B</code>
T6 (SUB)	<code>0011_1100_1111</code>	<code>~La</code> , <code>Eu</code> , <code>Su</code>	<code>A ← A-B</code>

A ideia central

A palavra de controle é a **receita de cada estado**: 12 bits que dizem exatamente *quem fala e quem ouve* no barramento naquele momento. O programa não muda essa receita — ela depende só do **estado (T1-T6)** e do **opcode**. É isso que faz o processador "funcionar sozinho".

Quem gera essa palavra a cada estado é o **controlador/sequenciador** (`controller_sequencer.v`), explicado em `FSM_explicacao.md`.

Como tudo funciona junto — o fluxo completo do SAP-1

Este é o documento que **junta as peças**: o barramento, a história de uma instrução do início ao fim, como o controlador decide, e um exercício pra você montar e traçar sozinho. Se você entender este documento, entendeu o SAP-1.

Pré-requisito: dá uma olhada em `componentes.md` (o que é cada bloco) e `palavra_controle.md` (os 12 sinais). Aqui a gente **conecta tudo**.

Parte 1 — O barramento e os "enables" (a base de tudo)

Um fio só para todo mundo

Todos os blocos estão ligados no **mesmo fio de 8 bits**: o **barramento W**. É como uma sala onde todos podem falar e ouvir — mas com uma regra:

Só UM bloco pode FALAR (colocar valor) no barramento por vez. Um ou mais podem OUVIR (copiar) ao mesmo tempo.

Se dois falassem juntos, os valores colidiriam (curto). Por isso o controlador tem o cuidado de ligar **exatamente um "falante"** em cada estado.

Quem pode falar × quem pode ouvir

Sinal	Tipo	Bloco	Ação
Ep	fala	PC	joga o endereço no barramento
~CE	fala	RAM	joga o dado lido no barramento
~Ei	fala	IR	joga o operando (endereço) no barramento
Ea	fala	Acumulador	joga A no barramento
Eu	fala	ALU	joga o resultado no barramento
~Lm	ouve	MAR	copia o barramento
~Li	ouve	IR	copia o barramento
~La	ouve	Acumulador	copia o barramento
~Lb	ouve	Registrador B	copia o barramento
~Lo	ouve	Saída	copia o barramento

(`Cp` = incrementa o PC e `Su` = modo da ALU não usam o barramento.)

Como é implementado de verdade (o mux)

No `sap1_top.v`, o barramento é um **multiplexador com prioridade** — não um tri-state. Só o "falante" habilitado passa:

```
w_bus = ep      ? {4'b0, pc_out}      : // PC falando
      (~ce)     ? ram_data           : // RAM falando
      (~ei)     ? {4'b0, ir_operand} : // IR falando
      ea        ? acc_out            : // A falando
      eu        ? alu_out            : // ALU falando
      8'b0;                                           // ninguém -> 0
```

Agora a palavra de controle faz sentido: cada estado escolhe **um bit de "falar"** e **um ou mais de "ouvir"**. É só isso que uma instrução faz — mover valores pelo barramento, um passo por vez.

Parte 2 — A história completa de uma instrução

Vamos seguir o **ADD 11** do programa 3×4, do primeiro ao último estado.

Situação inicial: o `LDA 11` anterior já rodou, então **A = 3**. O PC aponta para o `ADD 11` (endereço 1). E `mem[11] = 3`.

Em cada estado: quem **fala**, quem **ouve**, e o **valor no barramento**.

Estado	Fala	Ouve	Barramento	Efeito
T1	PC (<code>Ep</code>)	MAR (<code>~Lm</code>)	<code>01</code>	$MAR \leftarrow 1$
T2	—	— (PC conta)	<code>00</code>	$PC \leftarrow 2$
T3	RAM (<code>~CE</code>)	IR (<code>~Li</code>)	<code>1B</code>	$IR \leftarrow \text{"ADD 11"}$
T4	IR operando (<code>~Ei</code>)	MAR (<code>~Lm</code>)	<code>0B</code>	$MAR \leftarrow 11$
T5	RAM (<code>~CE</code>)	B (<code>~Lb</code>)	<code>03</code>	$B \leftarrow 3$
T6	ALU (<code>Eu</code>)	A (<code>~La</code>)	<code>06</code>	$A \leftarrow 3+3 = 6$

Lendo em português:

1. **T1 — "onde está a instrução?"** O PC vale 1. `Ep` liga o PC no barramento (aparece `01`); `~Lm` manda o MAR copiar. Agora **MAR = 1**.
2. **T2 — "adianta o PC."** `Cp` incrementa: **PC = 2** (já aponta pro próximo). O barramento não é usado.

3. **T3 — "lê a instrução."** O MAR (=1) faz a RAM entregar `mem[1] = ADD 11 ( 0x1B )`. `~CE` põe isso no barramento; `~Li` manda o IR copiar. Agora o **IR** tem a instrução, e o controlador enxerga `opcode = ADD`.
4. **T4 — "onde está o dado?"** O IR tem o operando 11. `~Ei` joga o 11 no barramento (0B); `~Lm` faz o MAR copiar. Agora **MAR = 11**.
5. **T5 — "pega o dado."** O MAR (=11) faz a RAM entregar `mem[11] = 3`. `~CE` põe no barramento (03); `~Lb` manda o **B** copiar. Agora **B = 3**.
6. **T6 — "soma!"** A ALU está sempre calculando `A + B = 3 + 3 = 6`. `Eu` põe esse 6 no barramento; `~La` manda o **A** copiar. Agora **A = 6**.

Fim: **A foi de 3 para 6**, gastando exatamente 6 estados. O PC já aponta pro próximo `ADD`, e o ciclo recomeça.

Note como **T1-T3 são sempre iguais** (buscar a instrução) e **T4-T6 mudam** conforme o opcode. É essa a divisão busca/execução.

Parte 3 — Como o controlador decide tudo isso

Quem ligou os enables certos em cada estado? O **controlador/sequenciador**. Ele tem duas partes:

(a) O contador de anel — "onde estamos"

Um registrador que percorre os estados em ordem, avançando na **borda de descida** do clock:

```
IDLE → T1 → T2 → T3 → T4 → T5 → T6 → T1 → ...
```

É só isso: um contador que dá voltas. Não depende de dado nenhum.

(b) A lógica combinacional — "o que fazer aqui"

Uma tabela (`case`) que, dado o **estado** e o **opcode**, produz a palavra de controle:

```
case (estado)
  T1: con = ...Ep, ~Lm...           // igual pra toda instrução
  T3: con = ...~CE, ~Li...
  T6: case (opcode)
    ADD: con = ...~La, Eu...        // A ← A+B
    SUB: con = ...~La, Eu, Su...    // A ← A-B
  endcase
endcase
```


A ideia-chave: a palavra de controle depende **só** do estado e do opcode — **nunca** do dado. Por isso o processador é **determinístico**: mesmo estado + mesmo opcode = mesmos sinais, sempre. É isso que faz ele "funcionar sozinho", sem ninguém mandar.

(Detalhes da FSM, HLT e reset em `FSM_explicacao.md`.)

Parte 4 — Exercício: monte e trace A = 5 + 2

Agora **você**. Vamos escrever um programa que calcula $5 + 2$ e mostra 7.

Passo 1 — planejar

Precisamos: carregar 5, somar 2, mostrar, parar. E guardar os dados (5 e 2) em algum endereço.

```
LDA 14 ; A ← 5    (o 5 está no endereço 14)
ADD 15 ; A ← 5+2  (o 2 está no endereço 15)
OUT    ; mostra 7
HLT    ; para
```

Passo 2 — montar (traduzir pra binário)

Lembre: cada palavra é `[opcode(4) | operando(4)]`. Opcodes: `LDA=0000` `ADD=0001` `OUT=1110` `HLT=1111`.

end	instrução	opcode	operando	
0	LDA 14	0000	1110	0000_1110
1	ADD 15	0001	1111	0001_1111
2	OUT	1110	0000	1110_0000
3	HLT	1111	0000	1111_0000
14	dado = 5	—	—	0000_0101
15	dado = 2	—	—	0000_0010

Passo 3 — traçar na mão (prever o resultado)

Siga só o acumulador A:

Instrução	A antes	A depois
LDA 14	0	5
ADD 15	5	7
OUT	7	7 (mostra 7)
HLT	7	7 (para)

Resultado esperado: **saída = 7** (0x07), travado em T4 com HLT.

Passo 4 — conferir na simulação

Cole no bloco de programa do `ram_16x8.v`:

```
mem[0] = 8'b0000_1110; // LDA 14
mem[1] = 8'b0001_1111; // ADD 15
mem[2] = 8'b1110_0000; // OUT
mem[3] = 8'b1111_0000; // HLT
mem[14] = 8'd5;
mem[15] = 8'd2;
```

Ajuste `RESULTADO_ESPERADO = 8'd7` em `tb_model/tb_sap1.v`, rode `do wave_sap1.do` e veja o `acc_out` fazer `0 → 5 → 7`. Se bateu com o seu traço à mão, **você entendeu de verdade**.

Desafio extra: mude `mem[15]` para 3 e preveja o resultado **antes** de simular.
(Resposta: 8.)

O modelo mental (resumo de tudo)

O SAP-1 é uma **máquina de estados** que, a cada passo (T1–T6), gera uma **palavra de controle** que escolhe **um bloco pra falar** e **um pra ouvir** no **barramento**. Uma instrução é só uma sequência fixa de 6 desses passos, movendo valores de um registrador para outro. O programa na RAM decide *quais* instruções, mas *como* cada uma funciona é sempre a mesma receita — ditada pelo estado e pelo opcode.

Se essa frase fizer sentido pra você, fechou.

A FSM do Controlador/Sequenciador do SAP-1

Este documento explica a máquina de estados (FSM) que comanda o SAP-1, acompanha os dois diagramas gerados (`fsm_diagram.*` e `fsm_exec_diagram.*`) e mostra um **exemplo rodando passo a passo**.

Código de referência: `controller_fsm.v` (versão FSM clássica) e `controller_sequencer.v` (versão contador de anel — equivalente).

1. O que a FSM faz

O SAP-1 não executa uma instrução "de uma vez". Cada instrução é quebrada em **micropassos**, um por estado de tempo (*T-state*). A FSM é só um contador que percorre esses estados em ordem e, **em cada estado**, liga os sinais de controle certos (a *palavra de controle* CON, de 12 bits).

São 6 estados de trabalho — **T1 a T6** — mais um estado **IDLE** de partida. Cada instrução leva exatamente 6 ciclos:

Fase	Estados	Faz o quê	Depende do opcode?
Busca (<i>fetch</i>)	T1, T2, T3	lê a instrução da RAM e coloca no IR	Não (igual p/ todas)
Execução (<i>execute</i>)	T4, T5, T6	faz o trabalho da instrução	Sim

É por isso que existem dois diagramas: `fsm_diagram` mostra o **anel de estados** (a "pista" que a FSM corre); `fsm_exec_diagram` mostra **o que cada instrução faz** dentro de T4-T6.

2. Os 3 blocos da FSM (formato clássico)

Em `controller_fsm.v` a máquina está escrita nos três processos canônicos:

- Registrador de estado** — guarda o estado atual (`state`). Avança na **borda de descida** do clock (`negedge clk`); o reset assíncrono (`~CLR=0`) leva para **IDLE** .
- Lógica de próximo estado** (combinacional) — a sequência fixa `IDLE → T1 → T2 → T3 → T4 → T5 → T6 → T1 → ...` . Quando o processador está parado (`run = 0`), a FSM **segura o estado** (congela).
- Lógica de saída** (combinacional, Moore) — gera a palavra CON a partir do estado (e, em T4-T6, também do opcode).

Por que a descida do clock?

A FSM troca de estado na **descida**, e os registradores do *datapath* (PC, A, B, IR, ...) carregam na **subida**. Assim, quando chega a subida, a palavra de controle daquele T-state **já está estável** — sem condição de corrida.

Por que o estado IDLE?

Ele "absorve a fase do reset": não importa em que ponto do clock você solte o `~CLR`, a máquina começa limpa e o primeiro `negedge` útil leva a T1.

Como o HLT para tudo (sem desligar o clock)

O sinal `halted` é travado em **T4 quando o opcode é HLT**. Ele faz `run = 0`, que é um **clock-enable**: a FSM para de avançar e a palavra de controle vira IDLE → nenhum registrador carrega → tudo congela. O clock continua oscilando (boa prática de FPGA — nada de *gating* de clock).

3. A palavra de controle (CON)

```
CON = { Cp, Ep, ~Lm, ~CE, ~Li, ~Ei, ~La, Ea, Su, Eu, ~Lb, ~Lo }
bit:  11 10  9  8  7  6  5  4  3  2  1  0
```

- Sinais **sem til** (Cp, Ep, Ea, Su, Eu): ativos em **ALTO** (1 = ativo).
- Sinais **com til** (~Lm, ~CE, ~Li, ~Ei, ~La, ~Lb, ~Lo): ativos em **BAIXO** (0 = ativo).

Significado rápido:

Sinal	Ação quando ativo
Ep	PC coloca seu valor no barramento
~Lm	MAR carrega do barramento
Cp	PC incrementa (+1)
~CE	RAM coloca o dado no barramento
~Li	IR carrega do barramento
~Ei	IR coloca o operando (endereço) no barramento
~La	Acumulador A carrega do barramento
Ea	A coloca seu valor no barramento
~Lb	Registrador B carrega do barramento
Eu	ALU coloca o resultado no barramento
Su	ALU subtrai (em vez de somar)
~Lo	Registrador de saída carrega do barramento

4. O ciclo de busca (T1-T3) — igual para toda instrução

Estado	Sinais ativos	Transferência	Em palavras
T1	Ep, ~Lm	MAR ← PC	"o endereço da próxima instrução vai para o MAR"
T2	Cp	PC ← PC + 1	"o PC já aponta para a instrução seguinte"
T3	~CE, ~Li	IR ← RAM[MAR]	"a instrução é lida e guardada no IR"

No fim de T3 o opcode já está no IR — então T4-T6 podem decidir o que fazer.

5. O ciclo de execução (T4-T6) — depende do opcode

Resumo do que cada instrução faz (ver detalhes em fsm_exec_diagram):

Instr.	Opcode	T4	T5	T6
LDA	0000	~Lm, ~Ei MAR←IR.end	~CE, ~La A←RAM	—
ADD	0001	~Lm, ~Ei MAR←IR.end	~CE, ~Lb B←RAM	~La, Eu A←A+B
SUB	0010	~Lm, ~Ei MAR←IR.end	~CE, ~Lb B←RAM	~La, Eu, Su A←A-B
OUT	1110	Ea, ~Lo OUT←A	—	—
HLT	1111	trava run←0	congelado	congelado

Onde um passo não faz nada (—), a FSM emite a palavra IDLE, mas **ainda gasta o ciclo** (todas as instruções duram 6 estados).

6. Exemplo rodando passo a passo

Vamos seguir as **3 primeiras instruções** do programa que está na RAM (ram_16x8.v), que calcula $3^3 = 27$. Endereço 12 contém o dado **3**.

```
end 0: LDA 12 ; A <- 3
end 1: ADD 12 ; A <- 3 + 3 = 6
end 2: ADD 12 ; A <- 6 + 3 = 9 (= 3^2)
```

Estado inicial após o reset: PC = 0, A = 0, B = 0.

Instrução 1 — LDA 12 (em PC=0)

T	Sinais	O que acontece	Estado depois
T1	Ep, ~Lm	MAR ← PC(0)	MAR=0
T2	Cp	PC ← 1	PC=1
T3	~CE, ~Li	IR ← RAM[0] = LDA 12	IR=0000_1100
T4	~Lm, ~Ei	MAR ← IR.end = 12	MAR=12

T	Sinais	O que acontece	Estado depois
T5	$\sim\text{CE}, \sim\text{La}$	$A \leftarrow \text{RAM}[12] = 3$	A=3
T6	(idle)	nada (LDA termina em 5 passos úteis)	—

→ Fim da instrução: **A = 3**, PC = 1.

Instrução 2 — ADD 12 (em PC=1)

T	Sinais	O que acontece	Estado depois
T1	$\text{Ep}, \sim\text{Lm}$	$\text{MAR} \leftarrow \text{PC}(1)$	MAR=1
T2	Cp	$\text{PC} \leftarrow 2$	PC=2
T3	$\sim\text{CE}, \sim\text{Li}$	$\text{IR} \leftarrow \text{RAM}[1] = \text{ADD } 12$	IR=0001_1100
T4	$\sim\text{Lm}, \sim\text{Ei}$	$\text{MAR} \leftarrow 12$	MAR=12
T5	$\sim\text{CE}, \sim\text{Lb}$	$B \leftarrow \text{RAM}[12] = 3$	B=3
T6	$\sim\text{La}, \text{Eu}$	$A \leftarrow A + B = 3 + 3$	A=6

→ Fim da instrução: **A = 6**, PC = 2.

Instrução 3 — ADD 12 (em PC=2)

T	Sinais	O que acontece	Estado depois
T1	$\text{Ep}, \sim\text{Lm}$	$\text{MAR} \leftarrow \text{PC}(2)$	MAR=2
T2	Cp	$\text{PC} \leftarrow 3$	PC=3
T3	$\sim\text{CE}, \sim\text{Li}$	$\text{IR} \leftarrow \text{RAM}[2] = \text{ADD } 12$	IR=0001_1100
T4	$\sim\text{Lm}, \sim\text{Ei}$	$\text{MAR} \leftarrow 12$	MAR=12
T5	$\sim\text{CE}, \sim\text{Lb}$	$B \leftarrow \text{RAM}[12] = 3$	B=3
T6	$\sim\text{La}, \text{Eu}$	$A \leftarrow A + B = 6 + 3$	A=9

→ Fim da instrução: **A = 9 = 3²**, PC = 3.

A próxima instrução do programa é OUT (PC=3), que copia A para o visor — mostrando **09** antes de seguir para a segunda metade do cálculo (chegando a $27 = 3^3$).

Resumo do exemplo

Após a instrução	A (decimal)	Significado
LDA 12	3	carregou o dado 3
ADD 12	6	$3 + 3$
ADD 12	9	$3 + 3 + 3 = 3^2$

Cada uma dessas instruções foi **6 estados da FSM** (T1-T6). O contador de anel simplesmente repete T1...T6 para cada instrução até encontrar o HLT, que congela tudo.

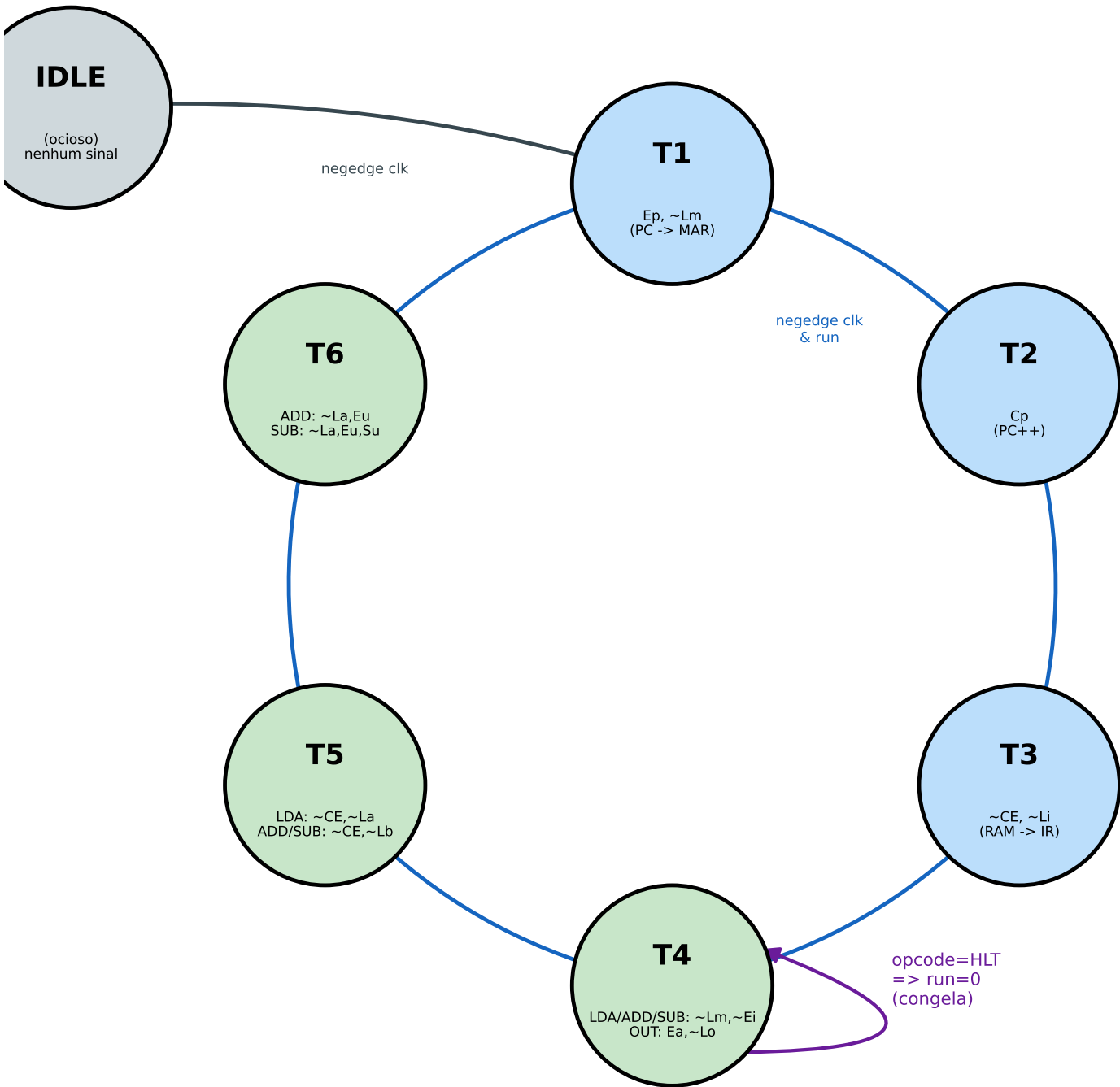
7. Como reproduzir os diagramas

```
python3 fsm_diagram.py      # anel de estados -> fsm_diagram.{png,pdf,svg}  
python3 fsm_exec_diagram.py # execução por instr -> fsm_exec_diagram.{png,pdf,svg}
```

Os `.pdf` / `.svg` são vetoriais — ideais para colar no relatório sem perder qualidade.

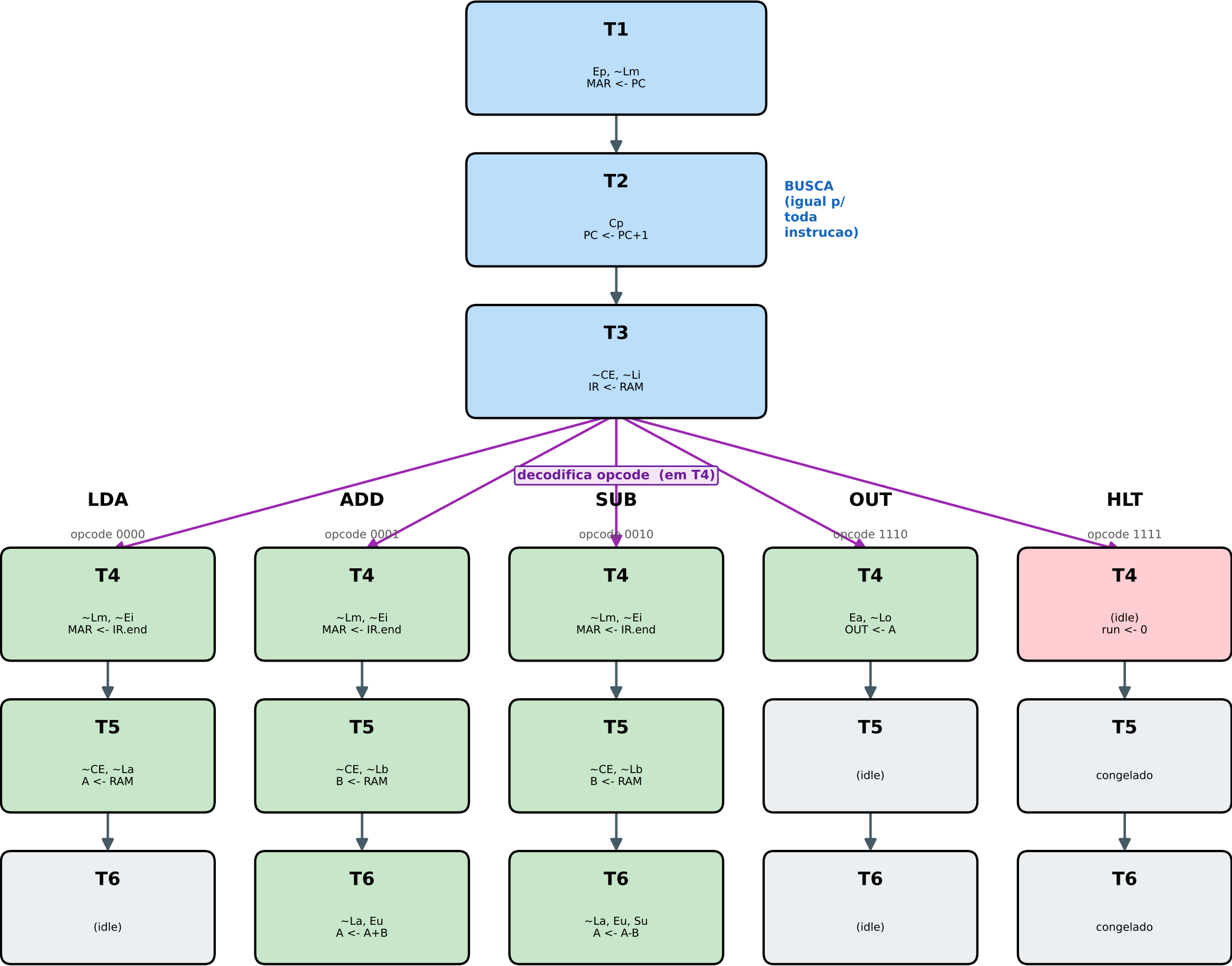
~CLR=0
(reset assinc.)

FSM do Controlador/Sequenciador - SAP-1



Azul = ciclo de BUSCA (T1-T3, igual p/ toda instrucao) Verde = ciclo de EXECUCAO (T4-T6, depende do opcode)
Estado avanca na borda de DESCIDA do clock; registradores carregam na SUBIDA.

Ciclo de Execucao por Instrucao - FSM do SAP-1

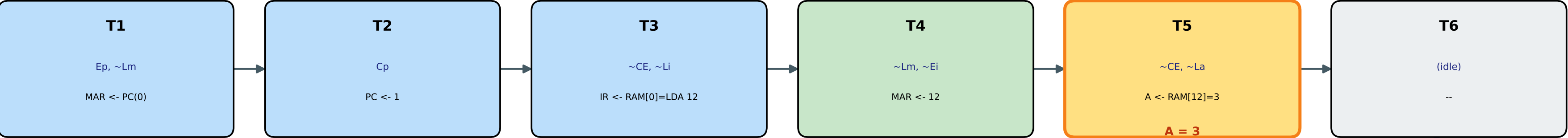


T6 -> T1: volta para a BUSCA da proxima instrucao (exceto HLT, que permanece congelado)

Verde = micropasso ativo Cinza = passo ocioso (palavra IDLE) Vermelho = HLT (run<-0)

LDA 12 (PC=0) A: 0 -> 3

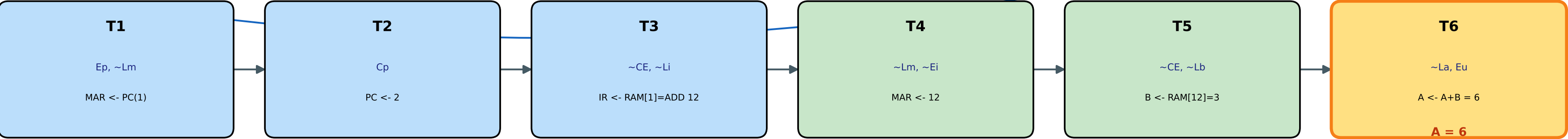
Trace da FSM - exemplo LDA 12 ; ADD 12 ; ADD 12 (A: 0->3->6->9)



A = 3

T6 -> T1
(proxima instr.)

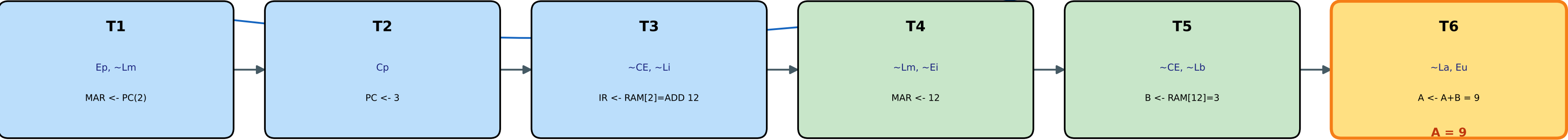
ADD 12 (PC=1) A: 3 -> 6



A = 6

T6 -> T1
(proxima instr.)

ADD 12 (PC=2) A: 6 -> 9 (= 3^2)



A = 9

Guia de leitura das ondas — SAP-1

Guia de referência para **estudar** as ondas (waveform) da simulação do SAP-1 no ModelSim. Não é roteiro de apresentação — é para você **entender** o que está vendo na tela.

Como abrir (na pasta `tb_model`):

```
do wave_sap1.do      # simulação completa (programa da RAM)
do wave_controller.do # só o controlador
```

O programa carregado na RAM é o **3 × 4 = 12** (somas repetidas de 3).

1. O que é cada sinal

Os sinais estão agrupados na janela Wave. Leia de cima para baixo:

Relógio / Reset

Sinal	O que é
<code>clk</code>	Clock do processador. Tudo acontece nas bordas dele.
<code>clr_bar</code>	Reset (~CLR), ativo em baixo : quando está em 0, zera tudo.

Controle

Sinal	O que é
<code>tstate</code>	Estado atual da máquina: IDLE, T1...T6. É o "onde estamos" no ciclo.
<code>opcode</code>	Instrução atual (LDA/ADD/SUB/OUT/HLT), lida da RAM no T3.
<code>hlt</code>	Sobe para 1 quando o processador para (instrução HLT).

Datapath (os registradores — em decimal)

Sinal	O que é
<code>pc_out</code>	PC (contador de programa): endereço da próxima instrução.
<code>addr_out</code>	MAR : endereço que está sendo lido da RAM agora.
<code>acc_out</code>	Acumulador A : onde a conta acontece. Siga esta linha!
<code>b_out</code>	Registrador B : segundo operando da soma/subtração.
<code>result</code>	Saída da ALU ($A \pm B$), combinacional.
<code>bus_w</code>	Barramento W : o "cano" por onde os dados trafegam (hex).

Saída

Sinal	O que é
out_port	Resultado — o que apareceria nos LEDs/displays. Só muda no OUT .

2. Como ler UM ciclo de instrução

Toda instrução leva **6 estados** (T1→T6). Os 3 primeiros são iguais para todas (busca); os 3 últimos dependem da instrução (execução).

Pegue a **primeira instrução** (LDA 11) e siga o tstate :

tstate	O que muda na onda	Significado
T1	addr_out (MAR) recebe o valor de pc_out	"pega o endereço da instrução"
T2	pc_out incrementa (0 → 1)	"PC já aponta para a próxima"
T3	opcode muda para LDA	"leu a instrução da RAM para o IR"
T4	addr_out recebe o operando (11)	"aponta para o dado"
T5	acc_out (A) recebe o dado (→ 3)	"carrega A com 3"
T6	nada muda (passo ocioso)	"LDA já terminou"

Depois o tstate volta para **T1** e começa a **próxima** instrução.

Regra de ouro: primeiro olhe o tstate (onde estamos), depois olhe qual registrador mudou naquele estado. Essa é a "história" da instrução.

3. Os detalhes que confundem (e a explicação)

"Por que o A do ADD só muda no T1 seguinte?"

No ADD , o sinal que carrega o acumulador (~La) fica ativo no **T6**. O registrador carrega na **borda de subida** que encerra o T6 — então o valor novo de acc_out só aparece **quando o tstate vira de T6 para T1**. Não é atraso da simulação; é o momento real da carga.

- LDA carrega A no **T5** → A muda entre T5 e T6.
- ADD / SUB carregam A no **T6** → A muda entre T6 e o T1 seguinte.

"Por que o resultado (out_port) fica parado?"

out_port é o **registrador de saída**. Ele só carrega quando roda a instrução **OUT** . Entre um OUT e outro ele **congela** — é normal. Para ver o A mudando, olhe acc_out , não out_port .

"Por que trava no T4 no final?"

O `HLT` é detectado no **T4**. Nesse ponto o `hlt` sobe para 1 e o contador de estados **para de avançar** — por isso o `tstate` fica preso em **T4** para sempre. O clock continua, mas nada mais carrega.

"Por que o estado avança na descida do clock?"

O contador de estados anda na **borda de descida**; os registradores do datapath carregam na **borda de subida**. Assim, quando chega a subida, a palavra de controle daquele estado já está estável. É de propósito.

4. Acompanhando o $3 \times 4 = 12$

Siga só a linha `acc_out` (**A**) ao longo da onda:

Instrução	A depois	Como
LDA 11	3	carrega o dado 3
ADD 11	6	$3 + 3$
ADD 11	9	$6 + 3$
ADD 11	12	$9 + 3$
OUT	<code>out_port</code> → 12	mostra o resultado
HLT	trava em T4, <code>hlt=1</code>	fim

$3 + 3 + 3 + 3 = 12 = 3 \times 4$. A multiplicação é feita por **somas repetidas**, porque o SAP-1 não tem instrução de multiplicar.

5. Checklist de autoestudo

Abra `do_wave_sap1.do`, dê zoom na primeira instrução e responda (as respostas estão na seção 2):

1. No **T1**, qual sinal muda? O que ele representa?
2. Em qual estado o `pc_out` incrementa?
3. Em qual estado o `opcode` passa a mostrar a instrução certa?
4. No **LDA**, em qual estado o `acc_out` recebe o valor?
5. No **ADD**, por que o `acc_out` novo só aparece no T1 seguinte?
6. O `out_port` muda em qual instrução?
7. Em qual estado o processador congela quando encontra o **HLT**?

Se você conseguir responder essas 7 olhando a onda, você **entendeu** o funcionamento do SAP-1.

Apêndice — ondas do controlador (`wave_controller.do`)

Essa onda mostra **só o controlador**, com os 12 sinais de controle. Útil para ver **quais sinais acendem em cada estado**:

- ativos-**alto** (`cp`, `ep`, `ea`, `su`, `eu`): acendem em **1**.
- ativos-**baixo** (`lm_bar`, `ce_bar`, `li_bar`, `ei_bar`, `la_bar`, `lb_bar`, `lo_bar`): acendem em **0**.

Exemplo: no **T1** você vê `ep=1` e `lm_bar=0` (PC → MAR). No **T3**, `ce_bar=0` e `li_bar=0` (RAM → IR).

Programa 2 explicado — a expressão (= 18)

Passo a passo de tudo que acontece no programa carregado na RAM: $(((7+3)-2)+5)-4+7-3+5 = 18$. Ele usa as 4 instruções (LDA, ADD, SUB, OUT) e mostra **dois** resultados: 9 no meio e 18 no fim.

Mapa da memória

end	binário	o que é	
0	0000 1011	LDA 11	PROGRAMA
1	0001 1100	ADD 12	
2	0010 1101	SUB 13	
3	0001 1110	ADD 14	
4	0010 1111	SUB 15	
5	1110 0000	OUT	
6	0001 1011	ADD 11	
7	0010 1100	SUB 12	
8	0001 1110	ADD 14	
9	1110 0000	OUT	
10	1111 0000	HLT	
11	0000 0111	= 7	DADOS
12	0000 0011	= 3	
13	0000 0010	= 2	
14	0000 0101	= 5	
15	0000 0100	= 4	

Como cada instrução roda (6 estados)

Toda instrução gasta 6 estados. Os 3 primeiros são **sempre iguais** (busca):

- **T1:** MAR $\leftarrow$ PC (pega o endereço da instrução)
- **T2:** PC $\leftarrow$ PC+1 (adianta o ponteiro)
- **T3:** IR $\leftarrow$ RAM (lê a instrução; o controlador passa a saber o opcode)

Os 3 últimos **dependem da instrução** (execução):

Instrução	T4	T5	T6
LDA n	MAR $\leftarrow$ n	A $\leftarrow$ RAM[n]	—
ADD n	MAR $\leftarrow$ n	B $\leftarrow$ RAM[n]	A $\leftarrow$ A+B
SUB n	MAR $\leftarrow$ n	B $\leftarrow$ RAM[n]	A $\leftarrow$ A-B
OUT	saída $\leftarrow$ A	—	—
HLT	congela	—	—

Trace completo — instrução por instrução

Estado inicial: **A=0, B=0, PC=0.**

#	PC	Instrução	O que faz	B	
1	0	LDA 11	$A \leftarrow \text{mem}[11]=7$	0	7
2	1	ADD 12	$B \leftarrow 3, A \leftarrow 7+3$	3	10
3	2	SUB 13	$B \leftarrow 2, A \leftarrow 10-2$	2	8
4	3	ADD 14	$B \leftarrow 5, A \leftarrow 8+5$	5	13
5	4	SUB 15	$B \leftarrow 4, A \leftarrow 13-4$	4	9
6	5	OUT	saída $\leftarrow A$	4	9 → mostra 09
7	6	ADD 11	$B \leftarrow 7, A \leftarrow 9+7$	7	16
8	7	SUB 12	$B \leftarrow 3, A \leftarrow 16-3$	3	13
9	8	ADD 14	$B \leftarrow 5, A \leftarrow 13+5$	5	18
10	9	OUT	saída $\leftarrow A$	5	18 → mostra 12 (hex de 18)
11	10	HLT	para em T4	5	18 (congelado)

A "história" do acumulador em duas metades: - **1ª metade** (até o 1º OUT): 7 → 10 → 8 → 13 → 9 = (((7+3)-2)+5)-4. - **2ª metade** (até o 2º OUT): continua de 9: → 16 → 13 → 18 = 9 + 7 - 3 + 5.

Os dois OUT

O OUT copia o acumulador **daquele momento** para o registrador de saída: - endereço 5: A=9 → mostra 09. - endereço 9: A=18 → mostra 12 (18 em hexadecimal).

Entre os dois, o visor **fica congelado em 9** — o registrador de saída só muda quando roda um OUT.

O HLT

No endereço 10, o HLT é detectado no **T4**, sobe hlt=1 e **congela o contador de estados em T4**. O clock continua, mas nada mais carrega.

O que ver na onda (do wave_sap1.do)

- **acc_out** (verde): a conta subindo/descendo — 7,10,8,13,9,16,13,18.
- **b_out**: muda a cada ADD/SUB (o operando: 3,2,5,4,7,3,5).
- **out_port** (laranja): pula pra 9, congela, depois pula pra 18.
- **opcode**: LDA, ADD, SUB, ADD, SUB, OUT, ADD, SUB, ADD, OUT, HLT.
- **hlt**: sobe no fim; **tstate** trava em T4.

Detalhes finos

1. **B é "descartável":** recarregado a cada ADD/SUB, não guarda resultado — é só o segundo número da conta do momento.
2. **A subtração liga Su=1 :** no T6 do SUB, além de `~La` e `Eu`, o `Su` ativa para a ALU subtrair.
3. **Dados são reusados:** `mem[11]=7` é lido na instrução 1 (LDA) e na 7 (ADD 11). O endereço é fixo; o dado fica lá para quem precisar.

Lista de exercícios — SAP-1 (com gabarito)

Responda **antes** de olhar o gabarito (no fim). Estudar respondendo é o que mais fixa. Se travar numa questão, o gabarito diz onde revisar.

Parte A — Componentes

- A1.** O que o PC (contador de programa) guarda? O que o sinal `Cp` faz?
- A2.** Qual a diferença entre o **MAR** e o **IR**? (o que cada um guarda?)
- A3.** A ALU **guarda** o resultado da soma? Por que, na onda, o sinal `result` fica "mudando" mesmo sem ninguém mandar?
- A4.** Por que o registrador B é chamado de "descartável"?

Parte B — O barramento e os enables

- B1.** Por que só **um** bloco pode "falar" (colocar valor) no barramento por vez?
- B2.** Cite os **5 sinais de "falar"** e o que cada um coloca no barramento.
- B3.** No estado T3, quem fala e quem ouve? Qual é a transferência?

Parte C — Palavra de controle

- C1.** Por que o estado de repouso (IDLE) é `0011_1110_0011` e **não** `0000_0000_0000` ?
- C2.** Decodifique `0010_1110_0001` : quais sinais estão ativos e qual a ação? (dica: compare com o IDLE)
- C3.** Decodifique `0011_1100_1111` : quais sinais e qual ação?

Parte D — Estados e ciclo

- D1.** Quantos estados uma instrução gasta? Quais são de **busca** e quais de **execução**?
- D2.** Em qual estado o PC incrementa? Em qual o IR é carregado?
- D3.** No `LDA` , em qual estado o acumulador A recebe o valor? E no `ADD` ? (por que é diferente?)
- D4.** Por que o processador congela em **T4** quando encontra o `HLT` ?

Parte E — Montar e traçar programas

- E1.** LDA 11 carrega o valor 3. Por quê? O que você mudaria para ele carregar 7?
- E2.** Escreva o binário (opcode|operando) destas instruções: LDA 13 , SUB 14 , OUT , HLT .
- E3.** Monte um programa que calcula $6 - 2 = 4$ e mostra o resultado. Dê o conteúdo das posições de memória usadas (em binário) e trace o acumulador.

Parte F — Ler a onda

- ✓**F1.** Na janela Wave, qual sinal você segue para "ver a conta acontecer"?
- F2.** Por que o out_port (resultado) fica **parado** entre dois OUT ?

Gabarito

- A1.** O PC guarda o **endereço da próxima instrução**. Cp faz o PC **incrementar** (+1) na próxima borda de subida. (rev.: componentes.md §1)
- A2.** O **MAR** guarda o **endereço** que a RAM vai ler (4 bits). O **IR** guarda a **instrução** lida (8 bits) e a divide em opcode + operando.
- A3.** Não. A ALU é **combinacional**: ela sempre mostra $A \pm B$ do momento. Como A e B mudam ao longo da execução, result acompanha — mas só vale quando Eu joga no barramento (no T6). (rev.: componentes.md §6)
- A4.** Porque ele é recarregado a cada ADD / SUB (no T5) e **não guarda resultado** — é só o "segundo número" da conta do momento.
- B1.** Porque é **um fio só**. Se dois falassem juntos, os valores colidiriam (curto/indefinido). Por isso o controlador liga só **um falante** por estado. (rev.: fluxo_completo.md, Parte 1)
- B2.** $E_p \rightarrow PC \cdot \sim CE \rightarrow RAM \cdot \sim Ei \rightarrow \text{operando do IR} \cdot Ea \rightarrow \text{acumulador} \cdot Eu \rightarrow \text{saída da ALU}$.
- B3.** No T3: a **RAM** fala ($\sim CE$) e o **IR** ouve ($\sim Li$). Transferência: **IR ← RAM** (busca da instrução).
- C1.** Porque há sinais **ativos-baixo** (com til): eles ficam em **1** quando desligados. Então o repouso tem os ativos-alto em 0 e os ativos-baixo em 1 → 0011_1110_0011 . (rev.: palavra_controle.md §2)
- C2.** $\sim CE = 0$ e $\sim Lb = 0 \rightarrow$ **RAM fala, B ouve** → **B ← RAM**. É o **T5 do ADD/SUB**.
- C3.** $\sim La = 0$, $Eu = 1$, $Su = 1 \rightarrow$ **A ouve, ALU fala, ALU subtrai** → **A ← A – B**. É o **T6 do SUB**.
- D1. 6 estados** (T1–T6). **Busca:** T1, T2, T3 (iguais p/ toda instrução). **Execução:** T4, T5, T6 (dependem do opcode).

D2. PC incrementa no **T2**. IR é carregado no **T3**.

D3. No `LDA`, A recebe no **T5**. No `ADD / SUB`, A recebe no **T6** (porque primeiro precisa carregar B no T5 e só então somar). Por isso, na onda, o A do ADD só muda na virada **T6 → T1 seguinte**.

D4. O `HLT` é detectado no T4 (o opcode já está no IR desde o T3). Nesse ponto trava `halted=1`, o contador de anel para de avançar e fica preso em **T4**. Não há o que fazer depois disso.

E1. Porque `11` é um **endereço**, não um valor — e na posição 11 está guardado o dado **3** (`mem[11]=3`). Para carregar 7, basta pôr `mem[11]=7` (ou apontar para um endereço que contenha 7). (*endereçamento direto*)

E2.

```
LDA 13 = 0000 1101
SUB 14 = 0010 1110
OUT   = 1110 0000
HLT   = 1111 0000
```

E3. Uma solução (dados em 14 e 15):

```
mem[0] = 0000_1110 ; LDA 14  -> A = 6
mem[1] = 0010_1111 ; SUB 15  -> A = 6-2 = 4
mem[2] = 1110_0000 ; OUT      -> mostra 4
mem[3] = 1111_0000 ; HLT
mem[14] = 0000_0110 ; dado = 6
mem[15] = 0000_0010 ; dado = 2
```

Traço do A: `0 → 6 → 4`. Resultado: **4** (`0x04`).

F1. A linha do **acumulador** `acc_out` (verde no `wave_sap1.do`). É onde a conta aparece subindo/descendo.

F2. Porque o `out_port` é o **registrador de saída**, que só carrega quando roda um `OUT` (`~Lo` ativo). Entre dois `OUT`, ele mantém o valor — por isso fica parado.

Desafios extras (sem gabarito — teste-se!)

1. Monte `2 × 5` por somas repetidas (resultado 10) e simule.
2. Decodifique `0101_1110_0011` e diga o estado. (*dica: é o T1*)
3. Sem simular, preveja a saída de: `LDA 15, ADD 15, ADD 15, OUT, HLT` com `mem[15]=4`. (*resposta: 12*)

Roteiro de apresentação — Simulação do SAP-1 ($3 \times 4 = 12$)

Comando para abrir a onda no ModelSim: `do wave_sap1.do`

Programa na RAM: $3 \times 4 = 12$ (multiplicação por somas repetidas de 3).

Cena 1 — Visão geral (Zoom Full)

Mostrar: a onda inteira.

Falar:

"Este é o SAP-1 executando 3×4 por somas repetidas. Em cima temos o clock e o reset; depois o estado do processador, a instrução atual e o sinal de parada (HLT); embaixo, os registradores (PC, MAR, acumulador A, registrador B, ALU) e a saída."

Apontar: a linha do `opcode` — a sequência de instruções `LDA → ADD → ADD → ADD → OUT → ... → HLT`. "Cada bloco é uma instrução."

Cena 2 — O ciclo de busca (zoom na 1ª instrução, LDA)

Fazer: dar **zoom** na primeira instrução (arrastar na régua de tempo). Agora dá pra ver T1 T2 T3 T4 T5 T6 no `tstate`.

Falar:

"Toda instrução começa igual: o ciclo de BUSCA, em três passos."

Apontar, estado a estado: - **T1** — `pc_out` vai para o `MAR` (`addr_out`). "Pega o endereço da instrução." - **T2** — `pc_out` incrementa ($0 \rightarrow 1$). "O PC já aponta para a próxima." - **T3** — `opcode` muda para `LDA`. "A instrução foi lida da RAM para o IR."

Cena 3 — A execução e a conta crescendo (acumulador A)

Apontar: a linha **verde** `acc_out` (**A**) — é a estrela.

Falar, acompanhando A:

"Agora a parte que faz a conta. Olhem o acumulador:" - LDA → **A = 3** ("carrega o primeiro 3") - ADD → **A = 6** ("soma mais 3") - ADD → **A = 9** - ADD → **A = 12** ("3 + 3 + 3 + 3 = 12, que é 3 × 4")

Detalhe fino (opcional, impressiona):

"Reparem que no ADD o A só muda na virada de T6 para o T1 seguinte — porque o sinal de carga do acumulador (~La) só fica ativo no T6, e o registrador carrega na borda que encerra esse estado."

Cena 4 — A saída (OUT)

Apontar: quando o `opcode` = **OUT**, a linha **laranja** `out_port` vai para **12**.

Falar:

"A instrução OUT copia o acumulador para o registrador de saída — é o que apareceria nos displays da placa. Aqui o resultado: 12."

Cena 5 — A parada (HLT)

Apontar: no fim, `opcode` = **HLT**, o `hlt` sobe para **1** e o `tstate` **congela em T4**.

Falar:

"Por fim, o HLT. Ele é detectado no estado T4 e congela o processador ali mesmo — o contador de estados para de avançar. Na placa, isso acende o LED de HLT e os displays ficam mostrando 0C, que é 12 em hexadecimal."

Cena 6 — Fechamento (resultado na placa)

Falar:

"Na DE10-Lite o painel final fica assim:"

HEX5	HEX4	HEX3 HEX2	HEX1 HEX0
4 (estado)	H (HLT)	0C (A=12)	0C (resultado)

"Estado 4, instrução H de HLT, acumulador e resultado em 0C — 12 decimal."

Resumo de 1 frase (para o final)

"O SAP-1 não multiplica de uma vez: ele soma o 3 quatro vezes, um passo de cada vez, controlado pela máquina de estados T1-T6 — e é exatamente isso que a simulação mostra."

Checklist técnico (se perguntarem)

- 6 estados por instrução (T1-T3 busca, T4-T6 execução).
- Estado avança na borda de DESCIDA; registradores carregam na SUBIDA.
- HLT por clock-enable (congela o anel), sem desligar o clock.
- Verificação: testbench autoverificável deu `PASSOU: saída = 12`.

Cola do SAP-1 (resumo de 1 página)

Instruções (5)

Instrução	Opcode	Faz
LDA n	0000	$A \leftarrow \text{memória}[n]$
ADD n	0001	$A \leftarrow A + \text{memória}[n]$
SUB n	0010	$A \leftarrow A - \text{memória}[n]$
OUT	1110	saída $\leftarrow A$
HLT	1111	para o processador

Formato da palavra (8 bits): [opcode(4) | operando(4)] O operando é um **endereço** (0-15), não um valor.

Os 6 estados

Fase	Estados	O que faz
Busca	T1, T2, T3	igual p/ toda instrução
Execução	T4, T5, T6	depende do opcode

- **T1:** $MAR \leftarrow PC$ · **T2:** $PC \leftarrow PC+1$ · **T3:** $IR \leftarrow RAM$
- Estado avança na **descida** do clock; registradores carregam na **subida**.

Palavra de controle (CON = 12 bits)

{ Cp, Ep, ~Lm, ~CE, ~Li, ~Ei, ~La, Ea, Su, Eu, ~Lb, ~Lo }
11 10 9 8 7 6 5 4 3 2 1 0

- **Ativo-alto** (1=liga): Cp Ep Ea Su Eu
- **Ativo-baixo** (0=liga): ~Lm ~CE ~Li ~Ei ~La ~Lb ~Lo
- **Repouso (IDLE):** 0011_1110_0011
- **Ler:** compare com o IDLE → os bits **diferentes** são os ativos.

Estado	CON	Ação
T1	0101_1110_0011	$MAR \leftarrow PC$
T2	1011_1110_0011	$PC \leftarrow PC+1$
T3	0010_0110_0011	$IR \leftarrow RAM$
T4 (LDA/ADD/SUB)	0001_1010_0011	$MAR \leftarrow \text{operando}$
T4 (OUT)	0011_1111_0010	saída $\leftarrow A$
T5 (LDA)	0010_1100_0011	$A \leftarrow RAM$

Estado	CON	Ação
T5 (ADD/SUB)	0010_1110_0001	$B \leftarrow \text{RAM}$
T6 (ADD)	0011_1100_0111	$A \leftarrow A+B$
T6 (SUB)	0011_1100_1111	$A \leftarrow A-B$

Quem fala / quem ouve no barramento

- **fala:** Ep (PC) ~CE (RAM) ~Ei (operando) Ea (A) Eu (ALU)
- **ouve:** ~Lm (MAR) ~Li (IR) ~La (A) ~Lb (B) ~Lo (saída)
- Regra: **um fala, um ou mais ouvem** por estado.

Displays na placa (HEX5 → HEX0)

HEX5	HEX4	HEX3 HEX2	HEX1 HEX0
estado T (1-6)	instrução (L/A/5/o/H)	acumulador A	resultado

Controles: SW0 =clock auto/manual · KEY1 =passo · KEY0 =reset · SW1 =debug.

Comandos ModelSim (na pasta tb_model)

Comando	Faz
do wave_sap1.do	simulação completa com ondas
do wave_controller.do	ondas do controlador
do run_one.do <nome>	um componente com ondas
do run_all_tb.do	todos os testes (PASSOU/FALHOU)

Fatos-chave

- Uma instrução = **6 estados**, sempre.
- O operando é **endereço**, não valor (endereçamento **direto**).
- A do LDA muda no T5; a do ADD / SUB muda no T6 (aparece no T1 seguinte).
- O **resultado** só muda no OUT ; o HLT congela em **T4**.
- Barramento = **mux** (um falante); HLT = **clock-enable** (sem gating).

Glossário — SAP-1

Termos-chave do projeto, cada um em uma linha. Bom para quem está começando.

Arquitetura

- **SAP-1** — *Simple-As-Possible computer*: processador didático de 8 bits do livro do Malvino.
- **Datapath** — o "caminho dos dados": os registradores e a ALU por onde os valores passam.
- **Barramento W** — o fio único de 8 bits que liga todos os blocos; só um bloco "fala" nele por vez.
- **Multiplexador (mux)** — o circuito que escolhe qual bloco fala no barramento (usado no lugar de tri-state, melhor para FPGA).

Blocos

- **PC** (Program Counter) — contador de 4 bits com o endereço da próxima instrução.
- **MAR** (Memory Address Register) — guarda o endereço que a RAM vai ler.
- **RAM 16×8** — memória de 16 posições de 8 bits (programa + dados); só leitura no SAP-1.
- **IR** (Instruction Register) — guarda a instrução atual e a divide em opcode + operando.
- **Acumulador (A)** — registrador principal, onde a conta acontece.
- **Registrador B** — segundo operando da soma/subtração.
- **ALU** — *Arithmetic Logic Unit*: faz $A + B$ ou $A - B$ (combinacional).
- **Registrador de saída** — guarda o resultado para os LEDs/displays.
- **Controlador/sequenciador** — o "cérebro" que gera os sinais de controle.

Instruções

- **Opcode** — os 4 bits que dizem **qual** instrução (LDA, ADD, ...).
- **Operando** — os 4 bits que dizem **onde** está o dado (um endereço).
- **Endereçamento direto** — o operando é o endereço do dado (por isso `LDA 11` = "carregue o que está no endereço 11").
- **LDA / ADD / SUB / OUT / HLT** — carregar / somar / subtrair / mostrar / parar.

Controle e tempo

- **Palavra de controle (CON)** — os 12 bits que ligam/desligam cada sinal em cada estado.
- **Ativo-alto** — sinal que age quando vale **1** (ex.: `Cp`, `Ep`).
- **Ativo-baixo** — sinal que age quando vale **0**; marcado com til `~` (ex.: `~La`).
- **Enable** — sinal que "habilita" um bloco (a falar ou a ouvir no barramento).
- **Ciclo de busca (fetch)** — T1-T3: pegar a instrução da memória.

- **Ciclo de execução (execute)** — T4–T6: fazer o trabalho da instrução.
- **T-state** — um dos estados de tempo (T1 a T6).
- **Contador de anel (ring counter)** — registrador que percorre os estados em círculo (T1→...→T6→T1).
- **One-hot** — codificação em que só um bit é 1 por vez (ex.: T3 = 000100).
- **Borda de subida/descida** — a transição do clock de 0→1 (subida) ou 1→0 (descida). Aqui: estados avançam na descida, registradores carregam na subida.
- **Clock-enable** — técnica de "congelar" um registrador sem desligar o clock (usada no HLT).
- **Gating de clock** — desligar o clock por lógica (má prática em FPGA; **evitada** aqui).
- **Reset assíncrono** — reset que age na hora, sem esperar o clock ($\sim$ CLR).

Simulação

- **Testbench** — código que aplica estímulos e verifica um módulo.
- **Autoverificável** — testbench que sozinho diz PASSOU/FALHOU.
- **Waveform (onda)** — o gráfico dos sinais ao longo do tempo (janela Wave).
- **.do** — script do ModelSim (comandos automatizados).
- **DUT** (*Device Under Test*) — o módulo sendo testado.